

Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

Section: C

Instructor Name: Miss Sabeen

Course: Data Structure & Algorithm Lab

Program: (BS) - Computer Science

Lab # 1

Task 1: Write a Program to enter three integers and output the smallest integers using IF.

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int arr[3];
    int size = sizeof(arr) / sizeof(int);

    cout << "Enter 3 integers: \n";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    int small = arr[0];

    if (small > arr[1]) {
        small = arr[1];
    }
    if (small > arr[2]) {
        small = arr[2];
    }

    cout << "The smallest integer is: " << small << endl;

    return 0;
}
```

Output:

Microsoft Visual Studio Debug Console

```
Enter 3 integers:
6
4
2
The smallest integer is: 2
```

Task 2: Write a Program to enter 10 integers in a single-dimension array and then print out the array in ascending order.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[10];
    int size = sizeof(arr) / sizeof(int);

    cout << "Enter 10 integers:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}
```

```

}

cout << "Array in ascending order: ";
for (int i = 0; i < size; i++) {
    int minIndex = 0;
    int minValue = INT_MAX;

    for (int j = 0; j < size; j++) {
        if (arr[j] < minValue) {
            minValue = arr[j];
            minIndex = j;
        }
    }

    cout << minValue << " ";

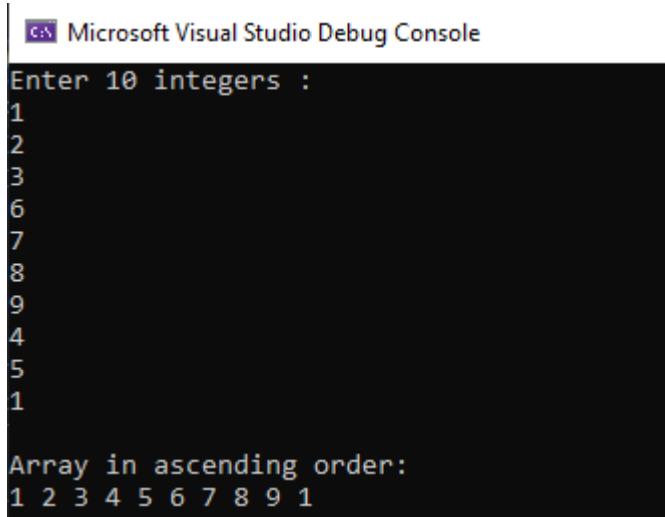
    arr[minIndex] = INT_MAX;
}

cout << endl;

return 0;
}

```

Output:



The screenshot shows the Microsoft Visual Studio Debug Console with the following text:

```

Enter 10 integers :
1
2
3
6
7
8
9
4
5
1

Array in ascending order:
1 2 3 4 5 6 7 8 9 1

```

Task 3: Write a program to create structure named student. Take information of student from user as input (StdID, StdName, StdAge etc.) Display the output.

Code:

```

#include <iostream>
#include<string>
using namespace std;

struct student
{
    string StdID;
    string StdName;
    int StdAge;
};

```

```

int main()
{
    student arr[3];

    cout << "Enter students detail: "<<endl;
    for (int i = 0;i < 3;i++) {
        cout << "Enter student " << i + 1 << " Id: ";
        cin.ignore();
        getline(cin, arr[i].StdID);

        cout << "Enter student " << i + 1 << " name: ";
        getline(cin, arr[i].StdName);

        cout << "Enter student " << i + 1 << " age: ";
        cin >> arr[i].StdAge;

        cout << endl;
    }

    cout << "Students detail: " << endl;
    for (int i = 0;i < 3;i++) {

        cout << "Student " << i + 1 << " Id: " << arr[i].StdID << endl;
        cout << "Student " << i + 1 << " name: " << arr[i].StdName << endl;
        cout << "Student " << i + 1 << " age: " << arr[i].StdAge << endl;
        cout << endl;
    }
    return 0;
}

```

Output:

 Microsoft Visual Studio Debug Console

```

Enter students detail:
Enter student 1 Id: 152
Enter student 1 name: Faisal
Enter student 1 age: 20

Enter student 2 Id: 123
Enter student 2 name: Qasim
Enter student 2 age: 20

Enter student 3 Id: 321
Enter student 3 name: Imran
Enter student 3 age: 20

Students detail:
Student 1 Id: 52
Student 1 name: Faisal
Student 1 age: 20

Student 2 Id: 123
Student 2 name: Qasim
Student 2 age: 20

Student 3 Id: 321
Student 3 name: Imran
Student 3 age: 20

```

Task 4: Perform above task using pointers to structure

Code:

```
#include <iostream>
#include<string>
using namespace std;

struct student
{
    string StdID;
    string StdName;
    int StdAge;
};

int main()
{
    student arr[3];

    cout << "Enter students detail: "<<endl;
    for (int i = 0;i < 3;i++) {
        student* ptr = &arr[i];
        cout << "Enter student " << i + 1 << " Id: ";
        cin.ignore();
        getline(cin, ptr->StdID);

        cout << "Enter student " << i + 1 << " name: ";
        getline(cin, ptr->StdName);

        cout << "Enter student " << i + 1 << " age: ";
        cin >> ptr->StdAge;

        cout << endl;
    }

    cout << "Students detail: " << endl;
    for (int i = 0;i < 3;i++) {

        cout << "Student " << i + 1 << " Id: " << arr[i].StdID << endl;
        cout << "Student " << i + 1 << " name: " << arr[i].StdName << endl;
        cout << "Student " << i + 1 << " age: " << arr[i].StdAge << endl;
        cout << endl;
    }
    return 0;
}
```

Output:

Microsoft Visual Studio Debug Console

```
Enter students detail:  
Enter student 1 Id: 152  
Enter student 1 name: Faisal  
Enter student 1 age: 20
```

```
Enter student 2 Id: 123  
Enter student 2 name: Khan  
Enter student 2 age: 21
```

```
Enter student 3 Id: 098  
Enter student 3 name: Imran  
Enter student 3 age: 22
```

```
Students detail:  
Student 1 Id: 52  
Student 1 name: Faisal  
Student 1 age: 20
```

```
Student 2 Id: 123  
Student 2 name: Khan  
Student 2 age: 21
```

```
Student 3 Id: 098  
Student 3 name: Imran  
Student 3 age: 22
```

Lab # 2

Task 1: Create a program that take an array of 10 inputs from the user and generate the sorted out put using the following Algorithms;

- Bubble Sort
- Insertion Sort
- Selection Sort

Bubble Sort

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int arr[10];
    int size = sizeof(arr) / sizeof(int);
    int swap, temp;

    cout << "Enter 10 integers : \n";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }
    for (int i = 0; i < size - 1; i++) {
        swap = 0;
        for (int j = 0; j < (size - i) - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swap = 1;
            }
        }
        if (swap == 0) {
            break;
        }
    }
    cout << "\nSorted output using Bubble Sort \n";
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:

```
Microsoft Visual Studio Debug Console

Enter 10 integers :
2
3
46
7
5
3
5
6
7
3

Sorted output using Bubble Sort
2 3 3 3 5 5 6 7 7 46
```

Insertion Sort

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int arr[10];
    int size = sizeof(arr) / sizeof(int);
    int temp,j=0;

    cout << "Enter 10 integers : \n";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }
    for (int i = 1;i < size ;i++) {
        temp = arr[i];
        j = i - 1;
        while (j>=0 && arr[j]>temp)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
    }
    cout << "\nSorted output using Insertion Sort\n";
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:

```
Microsoft Visual Studio Debug Console

Enter 10 integers :
1
2
6
7
8
0
3
4
5
9

Sorted output using Insertion Sort
0 1 2 3 4 5 6 7 8 9
```

Selection Sort

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int arr[10];
    int size = sizeof(arr) / sizeof(int);
    int min, loc, temp;

    cout << "Enter 10 integers : \n";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }
    for (int i = 0; i < size-1 ; i++) {
        min = arr[i];
        loc=i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < min) {
                min = arr[j];
                loc = j;
            }

            temp = arr[i];
            arr[i] = arr[loc];
            arr[loc] = temp;
        }
        cout << "\nSorted output using Selection Sort\n";
        for (int i = 0; i < size; i++)
        {
            cout << arr[i] << " ";

        }
        return 0;
    }
}
```

Output:

Microsoft Visual Studio Debug Console

Enter 10 integers :

1
2
3
6
7
8
94
6
7
8

Sorted output using Selection Sort

1 2 3 6 6 7 7 8 8 94

Lab # 3

Task 1: Translate the linear search algorithm in to a program which either find the location LOC where ITEM appears in ARRAY or return LOC=0

Code:

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int item, int size) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == item) {
            return i + 1;
        }
    }
    return 0;
}

int main() {
    int arr[] = { 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5 };
    int size = sizeof(arr) / sizeof(int);
    int x, r;

    cout << "Enter the number to search: ";
    cin >> x;

    r= linearSearch(arr, x, size);

    if (r != 0) {
        cout << x << " found at location " << r << endl;
    }
    else {
        cout << x << " not found in the array" << endl;
    }
    return 0;
}
```

Output:

Microsoft Visual Studio Debug Console

```
Enter the number to search: 4
4 found at location 3
```

Task 2: Translate binary search and insertion algorithm in to a program which finds either the location LOC where ITEM appears in ARRAY or the location LOC where ITEM should be inserted into ARRAY. (For object 3, 4, and 5 take an array of 10 elements given below)

Code:

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int BinarySearchInsertion(int arr[], int target, int size) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int mid = (right + left) / 2;

        if (arr[mid] == target) {
            return mid; // Item found, return its location
        }
        else if (arr[mid] < target) {
            left = mid + 1;
        }
    }
}
```

```

    }
    else {
        right = mid - 1;
    }
}
return left; // Item not found, return the location for insertion
}

int main() {
    int x;
    int arr[10];
    int size = sizeof(arr) / sizeof(int);

    cout << "Enter 10 integers: " << endl;
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    sort(arr, arr + size);

    cout << "Enter the number to search or insert: ";
    cin >> x;

    int result = BinarySearchInsertion(arr, x, size);

    if (arr[result] == x) {
        cout << x << " found at index: " << result << endl;
    }
    else {
        cout << x << " should be inserted at index: " << result << endl;
    }

    return 0;
}

```

Output:

Microsoft Visual Studio Debug Console

```

Enter 10 integers:
2
3
4
5
6
7
1
4
5
6
Enter the number to search or insert: 1
1 found at index: 0

```

Microsoft Visual Studio Debug Console

```

Enter 10 integers:
3
4
5
6
2
1
5
7
8
6
Enter the number to search or insert: 9
9 should be inserted at index: 10

```

Task 3: Write a program which uses Binary Search to search the elements 52, and 33 and print it.

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

int BinarySearch(int arr[], int size, int target) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid; // Item found, return its location
        }
        else if (arr[mid] < target) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
    }
    return -1; // Item not found
}

int main() {
    int arr[] = { 12, 22, 33, 44, 52, 62, 72, 82, 92 };
    int size = sizeof(arr) / sizeof(arr[0]);

    int target1 = 52;
    int target2 = 33;

    int result1 = BinarySearch(arr, size, target1);
    int result2 = BinarySearch(arr, size, target2);

    if (result1 != -1) {
        cout << target1 << " found at index: " << result1 << endl;
    }
    else {
        cout << target1 << " not found in the array" << endl;
    }

    if (result2 != -1) {
        cout << target2 << " found at index: " << result2 << endl;
    }
    else {
        cout << target2 << " not found in the array" << endl;
    }

    return 0;
}
```

Output:

Microsoft Visual Studio Debug Console

```
52 found at index: 4
33 found at index: 2
```

Task 4: Write a program which uses Binary Search to search the elements 45 and 78 and delete it. NUMBERS=[11,22,33,36,45,52,57,60,64,78]

Code:

```
#include <iostream>
using namespace std;

int binarySearchAndDelete(int arr[], int target, int& size) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            // Element found, delete it
            for (int i = mid; i < size - 1; ++i) {
                arr[i] = arr[i + 1];
            }
            size--; // Decrease the size of the array
            return mid;
        }
        else if (arr[mid] < target) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
    }
    return -1; // Element not found
}

int main() {
    int target1 = 45;
    int target2 = 78;
    int arr[10] = { 11, 22, 33, 36, 45, 52, 57, 60, 64, 78 };
    int size = sizeof(arr) / sizeof(int);

    int result1 = binarySearchAndDelete(arr, target1, size);
    int result2 = binarySearchAndDelete(arr, target2, size);

    if (result1 == -1) {
        cout << target1 << " was not found in the array" << endl;
    }
    else {
        cout << target1 << " found at index " << result1 << " and deleted." << endl;
    }

    if (result2 == -1) {
        cout << target2 << " was not found in the array" << endl;
    }
    else {
        cout << target2 << " found at index " << result2 << " and deleted." << endl;
    }

    cout << "Updated array after deletion: ";
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

 Microsoft Visual Studio Debug Console

```
45 found at index 4 and deleted.
78 found at index 8 and deleted.
Updated array after deletion: 11 22 33 36 52 57 60 64
```

Lab # 4

Task 1: Write a program as follows

----- Linked List implementation of LIST ADT -----

1. Insert an element at the end
2. Delete an element from end
3. Search an element
4. Display List
5. Is List Empty

Exit

Code:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int value) : data(value), next(nullptr) {}
};

void insertAtEnd(Node*& head, int value) {
    Node* newNode = new Node(value);

    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* current = head;
    while (current->next != nullptr) {
        current = current->next;
    }

    current->next = newNode;
}

void deleteFromEnd(Node*& head) {
    if (head == nullptr) {
        cout << "List is empty. Cannot delete.\n";
        return;
    }

    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;
    }

    Node* current = head;
    Node* previous = nullptr;

    while (current->next != nullptr) {
        previous = current;
        current = current->next;
    }

    delete current;
```

```

    previous->next = nullptr;
}

int searchElement(Node* head, int value) {
    Node* current = head;
    int position = 1;

    while (current != nullptr) {
        if (current->data == value) {
            return position;
        }

        current = current->next;
        position++;
    }

    return -1;
}

void displayList(Node* head) {
    cout << "List: ";
    Node* current = head;

    while (current != nullptr) {
        cout << current->data << " -> ";
        current = current->next;
    }

    cout << "NULL\n";
}

bool isListEmpty(Node* head) {
    return head == nullptr;
}

int main() {
    Node* head = nullptr;
    int choice, element, position;

    do {
        cout << "\n1. Insert at End\n";
        cout << "2. Delete from End\n";
        cout << "3. Search an Element\n";
        cout << "4. Display List\n";
        cout << "5. Is List Empty\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter the element to insert: ";
                cin >> element;
                insertAtEnd(head, element);
                break;

            case 2:
                deleteFromEnd(head);
                break;

            case 3:
                cout << "Enter the element to search: ";
                cin >> element;
                position = searchElement(head, element);
                if (position != -1) {
                    cout << "Element found at position " << position << "\n";
                }
                else {
                    cout << "Element not found in the list\n";
                }
            }
        }
    } while (choice != 6);
}

```

```

    }
    break;

case 4:
    displayList(head);
    break;

case 5:
    if (isListEmpty(head)) {
        cout << "The list is empty.\n";
    }
    else {
        cout << "The list is not empty.\n";
    }
    break;

case 6:
    cout << "Exiting the program.\n";
    break;

default:
    cout << "Invalid choice. Please enter a valid option.\n";
}
} while (choice != 6);

while (head != nullptr) {
    Node* temp = head;
    head = head->next;
    delete temp;
}

return 0;
}

```

Output:

```

1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 1
Enter the element to insert: 2

```

```

1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 3
Enter the element to search: 4
Element not found in the list

```

```

1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 4
List: 2 -> 3 -> 4 -> NULL

```

```

1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 3
Enter the element to search: 3
Element found at position 2

```

```

1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 2

```

```

1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 5
The list is not empty.

```

```
1. Insert at End
2. Delete from End
3. Search an Element
4. Display List
5. Is List Empty
6. Exit
Enter your choice: 6
Exiting the program.
```

Lab # 5

Task 1: Write a program as follows for STACK

-----Array based implementation of STACK-----

1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit

Please Enter Your Choice:

Code:

```
#include <iostream>
using namespace std;

const int N= 10;
int stack[N];
int top = -1;
int total = 0;

void push(int x)
{
    if (top == N-1) {
        cout << "stack overflow";
        return;
    }
    else {
        top++;
        stack[top] = x;
    }
}

void pop() {
    if (top == -1) {
        cout << "No element to pop";
    }
    else {
        top--;
    }
}

void Display() {
    int i;
    cout << "Stack: ";
    for (int i = 0; i <= top; i++) {
        cout << " " << stack[i];
    }
    cout << endl;
}

void top_element() {
    if (top == -1) {
        cout << "Stack is empty. No top element" << endl;
    }
    else {
        cout << "Top element is: " << stack[top] << endl;
    }
}

int main()
{
    int choice, x;
    cout << "-----Array based implementation of STACK-----" <<
endl;

    do {
```

```

    cout << "\n1. Push an element on stack\n";
    cout << "2. Pop an element from stack\n";
    cout << "3. Display all\n";
    cout << "4. Top element\n";
    cout << "5. Exit\n";
    cout << "-----"
<< endl;
    cout << "\nPlease Enter Your Choice: ";
    cin >> choice;

    switch (choice) {
    case 1:
        cout << "\nEnter an element to push on stack: ";
        cin >> x;
        push(x);
        break;
    case 2:
        pop();
        break;

    case 3:
        Display();
        break;

    case 4:
        top_element();
        break;

    case 5:
        cout << "Exiting the program.\n";
        break;

    default:
        cout << "Invalid choice. Please enter a valid option.\n";
    }
    while (choice != 5);

    return 0;
}

```

Output:

 D:\sir syed(3)\Data Structure and algorithm Lab\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe

-----Array based implementation of STACK-----

```

1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit

```

Please Enter Your Choice: 1

Enter an element to push on stack: 4

```

1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit

```

Please Enter Your Choice: 3

Stack: 4 3 5

```
1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit
-----
```

```
Please Enter Your Choice: 2
```

```
1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit
-----
```

```
Please Enter Your Choice: 4
Top element is: 3
```

```
1. Push an element on stack
2. Pop an element from stack
3. Display all
4. Top element
5. Exit
-----
```

```
Please Enter Your Choice: 5
Exiting the program.
```

Task 2: Test the program using the following procedure: STACK of size N=6

1. Call PUSH(5)
2. Call PUSH(2)
3. Call PUSH(3)
4. Call POP()
5. Call PUSH(6)
6. Call PUSH(9)
7. Call PUSH(3)
8. Call DISPLAY()
9. Call TOP()

Code:

```
#include <iostream>
using namespace std;

const int N= 6;
int stack[N];
int top = -1;

void push(int x) {
    if (top == N - 1) {
        cout << "Stack overflow\n";
    }
    else {
        top++;
        stack[top] = x;
        cout << x << " pushed onto the stack.\n";
    }
}

void pop() {
```

```

    if (top == -1) {
        cout << "Stack is empty. Cannot pop.\n";
    }
    else {
        cout << stack[top] << " popped from the stack.\n";
        top--;
    }
}

void display() {
    if (top == -1) {
        cout << "Stack is empty.\n";
    }
    else {
        cout << "Stack: ";
        for (int i = 0; i <= top; i++) {
            cout << stack[i] << " ";
        }
        cout << endl;
    }
}

void topElement() {
    if (top == -1) {
        cout << "Stack is empty. No top element.\n";
    }
    else {
        cout << "Top element is: " << stack[top] << endl;
    }
}

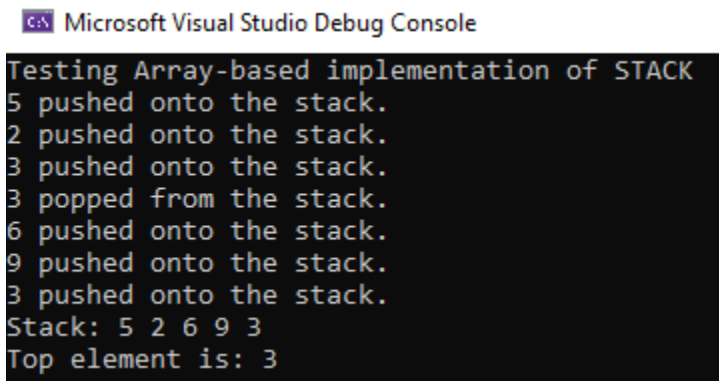
int main() {
    cout << "Testing Array-based implementation of STACK\n";

    push(5);
    push(2);
    push(3);
    pop();
    push(6);
    push(9);
    push(3);
    display();
    topElement();

    return 0;
}

```

Output:



Microsoft Visual Studio Debug Console

```

Testing Array-based implementation of STACK
5 pushed onto the stack.
2 pushed onto the stack.
3 pushed onto the stack.
3 popped from the stack.
6 pushed onto the stack.
9 pushed onto the stack.
3 pushed onto the stack.
Stack: 5 2 6 9 3
Top element is: 3

```

Lab # 6

Task 1: Write a program as follows for QUEUE

-----Array based implementation of QUEUE -----

1. Enqueue an element to queue
2. Dequeue an element from queue
3. Display all
4. Exit

Please Enter Your Choice:

Code:

```
#include <iostream>
using namespace std;

const int N= 10;
int queue[N];
int front = -1;
int rear = -1;

void enqueue(int x) {
    if (rear == N-1) {
        cout << "Queue is full. Cannot enqueue.\n";
    }
    else {
        if (front == -1) {
            front = 0;
        }
        rear++;
        queue[rear] = x;
        cout << x << " enqueued to the queue.\n";
    }
}

void dequeue() {
    if (front == -1) {
        cout << "Queue is empty. Cannot dequeue.\n";
    }
    else {
        cout << queue[front] << " dequeued from the queue.\n";
        front++;
        if (front > rear) {
            front = rear = -1;
        }
    }
}

void display() {
    if (front == -1) {
        cout << "Queue is empty.\n";
    }
    else {
        cout << "Queue: ";
        for (int i = front; i <= rear; i++) {
            cout << queue[i] << " ";
        }
        cout << endl;
    }
}

int main() {
    int choice, x;

    cout << "-----Array based implementation of QUEUE-----" <<
endl;
```

```

do {
    cout << "\n1. Enqueue an element to queue\n";
    cout << "2. Dequeue an element from queue\n";
    cout << "3. Display all\n";
    cout << "4. Exit\n";
    cout << "-----"
<< endl;
    cout << "\nPlease Enter Your Choice: ";
    cin >> choice;

    switch (choice) {
    case 1:
        cout << "Enter an element to enqueue: ";
        cin >> x;
        enqueue(x);
        break;

    case 2:
        dequeue();
        break;

    case 3:
        display();
        break;

    case 4:
        cout << "Exiting the program.\n";
        break;

    default:
        cout << "Invalid choice. Please enter a valid option.\n";
    }
} while (choice != 4);

return 0;
}

```

Output:

 D:\sir syed(3)\Data Structure and algorithm Lab\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe

```

-----Array based implementation of QUEUE-----
1. Enqueue an element to queue
2. Dequeue an element from queue
3. Display all
4. Exit
-----
Please Enter Your Choice: 1
Enter an element to enqueue: 2
2 enqueued to the queue.

```

```

1. Enqueue an element to queue
2. Dequeue an element from queue
3. Display all
4. Exit
-----
Please Enter Your Choice: 2
2 dequeued from the queue.

```

```
1. Enqueue an element to queue
2. Dequeue an element from queue
3. Display all
4. Exit
```

```
Please Enter Your Choice: 3
Queue: 3 5
```

```
1. Enqueue an element to queue
2. Dequeue an element from queue
3. Display all
4. Exit
```

```
Please Enter Your Choice: 4
Exiting the program.
```

Task 2: Test the program using the following procedure: QUEUE of size N=6

Call enqueue(5)
Call enqueue(2)
Call enqueue(3)
Call dequeue()
Call dequeue()
Call enqueue(6)
Call enqueue(3)
Call Display()

Code:

```
#include <iostream>
using namespace std;

const int N= 6;
int queue[N];
int front = -1;
int rear = -1;

void enqueue(int x) {
    if (rear == N-1) {
        cout << "Queue is full. Cannot enqueue.\n";
    }
    else {
        if (front == -1) {
            front = 0;
        }
        rear++;
        queue[rear] = x;
        cout << x << " " << "enqueued to the queue.\n";
    }
}

void dequeue() {
    if (front == -1) {
        cout << "Queue is empty. Cannot dequeue.\n";
    }
    else {
        cout << queue[front] << " " << "dequeued from the queue.\n";
        front++;
        if (front > rear) {
            front = rear = -1;
        }
    }
}

void display() {
```

```

if (front == -1) {
    cout << "Queue is empty.\n";
}
else {
    cout << "Queue: ";
    for (int i = front; i <= rear; i++) {
        cout << queue[i] << " ";
    }
    cout << endl;
}
}

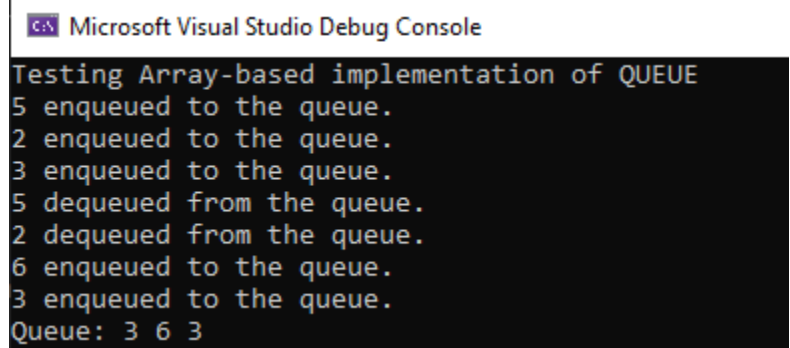
int main() {
    cout << "Testing Array-based implementation of QUEUE\n";

    enqueue(5);
    enqueue(2);
    enqueue(3);
    dequeue();
    dequeue();
    enqueue(6);
    enqueue(3);
    display();

    return 0;
}

```

Output:



Microsoft Visual Studio Debug Console

```

Testing Array-based implementation of QUEUE
5 enqueued to the queue.
2 enqueued to the queue.
3 enqueued to the queue.
5 dequeued from the queue.
2 dequeued from the queue.
6 enqueued to the queue.
3 enqueued to the queue.
Queue: 3 6 3

```

Lab # 7

Task 1: Write a program to create a Tree for the following.
Insert Sequence of data: 52, 25, 60, 35, 20, 59, 65

Code:

```
#include <iostream>
using namespace std;

struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

TreeNode* insert(TreeNode* node, int value) {
    if (node == nullptr) {
        return new TreeNode(value);
    }

    if (value < node->data) {
        node->left = insert(node->left, value);
    }
    else if (value > node->data) {
        node->right = insert(node->right, value);
    }

    return node;
}

void inOrderTraversal(TreeNode* node) {
    if (node != nullptr) {
        inOrderTraversal(node->left);
        cout << node->data << " ";
        inOrderTraversal(node->right);
    }
}

int main() {
    TreeNode* root = nullptr;

    root = insert(root, 52);
    insert(root, 25);
    insert(root, 60);
    insert(root, 35);
    insert(root, 20);
    insert(root, 59);
    insert(root, 65);

    cout << "In-order traversal of the tree: ";
    inOrderTraversal(root);
    cout << std::endl;
```

```
    return 0;  
}
```

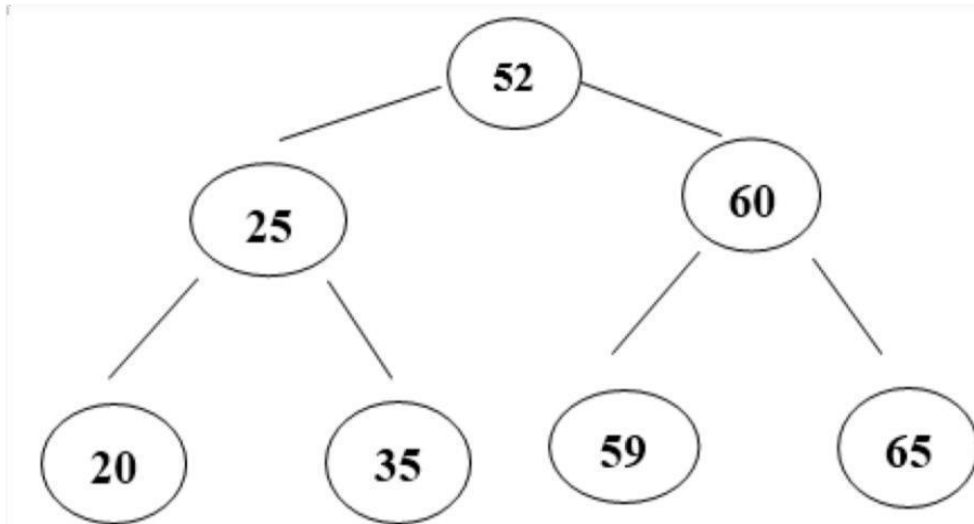
Output:

 Microsoft Visual Studio Debug Console

```
In-order traversal of the tree: 20 25 35 52 59 60 65
```

Lab # 8

Task 1: Write a program to to print the nodes of the following Tree diagram through Pre-Order, InOrder. and Post-Order.



Code:

```
#include<iostream>
using namespace std;

struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int value) : data(value), left(nullptr), right(nullptr) {}
};

// PreOrder traversal - visit the parent first and then left and right children;
void preOrderTraversal(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    cout << root->data << ", ";
    preOrderTraversal(root->left);
    preOrderTraversal(root->right);
}

// InOrder traversal - visit the left child, then the parent and the right child;
void inOrderTraversal(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    inOrderTraversal(root->left);
    cout << root->data << ", ";
    inOrderTraversal(root->right);
}

// PostOrder traversal - visit left child, then the right child and then the parent;
void postOrderTraversal(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    postOrderTraversal(root->left);
    postOrderTraversal(root->right);
    cout << root->data << ", ";
}
```

```

TreeNode* insert(TreeNode* root, int data) {
    if (root == nullptr) {
        return new TreeNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    }
    else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

int main() {
    TreeNode* root = nullptr;

    root = insert(root, 52);
    root = insert(root, 25);
    root = insert(root, 60);
    root = insert(root, 20);
    root = insert(root, 35);
    root = insert(root, 59);
    root = insert(root, 65);

    cout << "We have Inserted: 52, 25, 60, 20, 35, 59, 65" << endl;

    cout << "\nPreorder: ";
    preOrderTraversal(root);

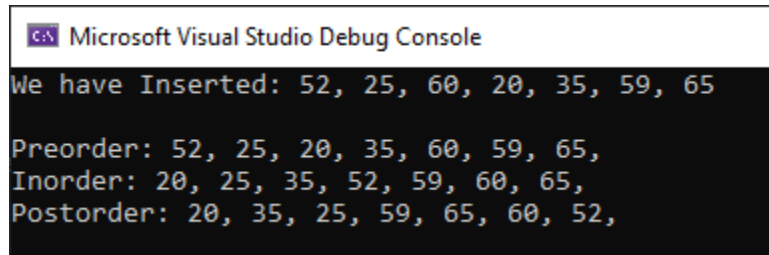
    cout << "\nInorder: ";
    inOrderTraversal(root);

    cout << "\nPostorder: ";
    postOrderTraversal(root);
    cout << std::endl;

    return 0;
}

```

Output:



```

C:\> Microsoft Visual Studio Debug Console

We have Inserted: 52, 25, 60, 20, 35, 59, 65

Preorder: 52, 25, 20, 35, 60, 59, 65,
Inorder: 20, 25, 35, 52, 59, 60, 65,
Postorder: 20, 35, 25, 59, 65, 60, 52,

```

Implementation of Graph and its representation**Task 1: Implement representation of graph by Adjacency Matrix and Adjacency List.****Code:**

```

#include <iostream>
#include <list>
using namespace std;

const int MAX_VERTICES = 50;

int AdjMatrix[MAX_VERTICES][MAX_VERTICES];

// This function will print Adjacency Matrix
void printAdjMatrix(int v) {
    cout << "\n\tAdjacency Matrix for undirected Graph:\n" << endl;

    for (int i = 0; i < v; i++) {
        for (int j = 0; j < v; j++) {
            cout << AdjMatrix[i][j] << " ";
        }
        cout << endl;
    }
}

// This function will mark the vertices which have edges between them
void addEdgeMatrix(int x, int y) {
    AdjMatrix[x][y] = AdjMatrix[y][x] = 1;
}

// This function will push element into a list from back
void addEdgeList(list<int>* l, int x, int y) {
    l[x].push_back(y);
    l[y].push_back(x);
}

// This function will print Adjacency List
void printAdjList(list<int>* l, int v) {
    cout << "\n\tAdjacency List for undirected Graph:\n" << endl;

    for (int i = 0; i < v; i++) {
        cout << i;
        for (auto nbr : l[i]) {
            cout << "-->" << nbr;
        }
        cout << endl;
    }
}

int main() {
    int v, e, x, y;

    cout << "\n\tEnter the number of vertices in Graph: ";
    cin >> v;

    cout << "\tEnter the number of edges in Graph: ";
    cin >> e;
    cout << "\n\t" << endl;

    // 1D array for adjacency list
    list<int>* graphList = new list<int>[v];

```

```

for (int i = 0; i < e; i++) {
    cout << "\tEnter edge number " << (i + 1) << ": ";
    cin >> x >> y;

    addEdgeMatrix(x, y);

    addEdgeList(graphList, x, y);
}

cout << endl;

printAdjMatrix(v);

cout << endl;

printAdjList(graphList, v);

delete[] graphList; // free memory allocated for adjacency list

return 0;
}

```

Output:

Microsoft Visual Studio Debug Console

```

Enter the number of vertices in Graph: 8
Enter the number of edges in Graph: 5

```

```

Enter edge number 1: 5 5
Enter edge number 2: 4 4
Enter edge number 3: 3 3
Enter edge number 4: 2 2
Enter edge number 5: 1 1

```

Adjacency Matrix for undirected Graph:

```

0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0
0 0 1 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0

```

Adjacency List for undirected Graph:

```

0
1-->1-->1
2-->2-->2
3-->3-->3
4-->4-->4
5-->5-->5
6
7

```

Implementation of Breadth first and Depth first Traversal for a Graph**Task 1: Implement Breadth first and Depth first Traversal for a Graph.****Code:**

```

#include <iostream>
#include <list>
#include <queue>

using namespace std;

class Graph {
    int V;
    list<int>* l;

public:
    Graph(int V) {
        this->V = V;
        l = new list<int>[V];
    }

    void AddEdge(int x, int y) {
        l[x].push_back(y);
    }

    void BreadthFirstTraversal(int S) {
        bool* visited = new bool[V];
        for (int i = 0; i < V; i++) {
            visited[i] = false;
        }

        queue<int> q;
        visited[S] = true;
        q.push(S);

        while (!q.empty()) {
            S = q.front();
            cout << S << " ";
            q.pop();

            for (auto i = l[S].begin(); i != l[S].end(); i++) {
                if (!visited[*i]) {
                    visited[*i] = true;
                    q.push(*i);
                }
            }
        }
    }

    void DFS_visit(int d, bool visited[]) {
        visited[d] = true;
        cout << d << " ";

        for (auto i = l[d].begin(); i != l[d].end(); i++) {
            if (!visited[*i]) {
                DFS_visit(*i, visited);
            }
        }
    }

    void DepthFirstTraversal(int d) {
        bool* visited = new bool[V];
        for (int i = 0; i < V; i++) {
            visited[i] = false;
        }
    }
}

```

```

    }

    DFS_visit(d, visited);
}

};

int main() {
    int v, e, x, y, k;

    cout << "Enter the number of vertices in Graph: ";
    cin >> v;

    cout << "Enter the number of edges in Graph: ";
    cin >> e;

    Graph g(v);

    for (int i = 0; i < e; i++) {
        cout << "Enter edge number " << (i + 1) << ": ";
        cin >> x >> y;
        g.AddEdge(x, y);
    }

    cout << "Enter source vertex: ";
    cin >> k;

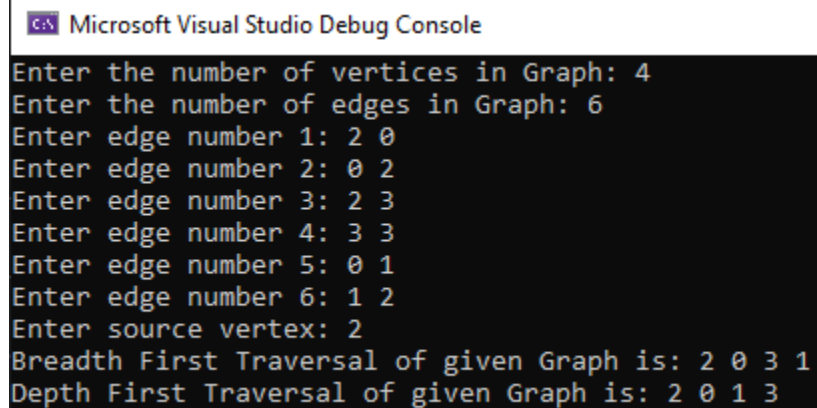
    cout << "Breadth First Traversal of given Graph is: ";
    g.BreadthFirstTraversal(k);
    cout << endl;

    cout << "Depth First Traversal of given Graph is: ";
    g.DepthFirstTraversal(k);
    cout << endl;

    return 0;
}

```

Output:



```

Microsoft Visual Studio Debug Console
Enter the number of vertices in Graph: 4
Enter the number of edges in Graph: 6
Enter edge number 1: 2 0
Enter edge number 2: 0 2
Enter edge number 3: 2 3
Enter edge number 4: 3 3
Enter edge number 5: 0 1
Enter edge number 6: 1 2
Enter source vertex: 2
Breadth First Traversal of given Graph is: 2 0 3 1
Depth First Traversal of given Graph is: 2 0 1 3

```